

Terraform

Project Structure Cheat Sheet

Terraform configuration files are where you write your Terraform code. They use the `.tf` extension and a declarative language called HashiCorp Configuration Language (HCL) to describe the components used to automate your infrastructure.

File types in Terraform projects

File type	Purpose	Editable by user
<code>.tf</code>	Core configuration	✅ Yes
<code>.tf.json</code>	JSON-based configuration	✅ Sometimes
<code>.tfvars</code>	Variable values	✅ Yes (treat as sensitive)
<code>.tfstate</code>	Current infrastructure state	❌ No (managed by Terraform)
<code>.tfstate.backup</code>	Backup of previous state	❌ No
<code>.terraform.lock.hcl</code>	Provider version lock	⚠️ Auto-generated — do commit it, rarely hand-edit
<code>.terraform/</code>	Local working data	❌ No (never commit)
<code>.terraformrc</code> / <code>terraform.rc</code>	CLI config	✅ Yes

Common files and patterns

File / Pattern	Purpose	Typical contents
<code>main.tf</code>	Primary configuration defining core resources (compute, networking, storage)	<code>resource</code> blocks like <code>resource "aws_instance" "web" { ... }</code> or <code>resource "azurerm_network_interface" "example" { ... }</code>
<code>variables.tf</code>	Declares input variables for reuse, parameterization, and portability	<code>variable</code> blocks with <code>type</code> , <code>description</code> , and optional <code>default</code> (e.g., <code>variable "region" { default = "us-east-1" }</code>)
<code>provider.tf</code>	Configures providers and version constraints for consistent deployments	<code>provider</code> blocks (e.g., <code>provider "aws" { region = var.region }</code>)
<code>output.tf</code>	Defines values printed after <code>apply</code> (IPs, names, URLs) for integration and pipelines	<code>output</code> blocks (e.g., <code>output "instance_ip" { value = aws_instance.web.public_ip }</code>)
<code>*.tfvars</code>	Variable values per environment. Auto-loaded if named <code>terraform.tfvars</code> or <code>*.auto.tfvars</code>	Key/value pairs overriding defaults (e.g., <code>instance_type = "t3.medium"</code>)
<code>README.md</code>	Project documentation and instructions	Setup steps, usage examples, and architecture notes
Automation scripts	CI/CD or helper scripts for repeatable workflows	Bash/PowerShell/Python that run <code>terraform fmt</code> , <code>init</code> , <code>validate</code> , <code>plan</code> , <code>apply</code>
YAML files	CI/CD pipelines or Kubernetes manifests that integrate with Terraform	<code>.github/workflows/*.yaml</code> , GitLab CI, or Kubernetes YAMLs

Terraform project structure example

```
terraform-project/
├── README.md
├── provider.tf
├── variables.tf
├── main.tf
├── output.tf
├── terraform.tfvars
├── envs/
│   ├── dev.tfvars
│   ├── staging.tfvars
│   └── prod.tfvars
├── modules/
│   ├── network/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── README.md
│   ├── compute/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── README.md
│   └── security/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── scripts/
│   ├── deploy.sh
│   ├── destroy.sh
│   ├── validate.sh
│   └── plan.sh
├── .github/
│   └── workflows/
│       ├── terraform-plan.yml
│       └── terraform-apply.yml
├── .terraform.lock.hcl
├── .gitignore
└── .terraform/                # Auto-generated working directory (not committed)
```

.gitignore file

`.gitignore` keeps your repository clean, secure, and portable by excluding files that must remain local (state, credentials, IDE files).

A solid baseline for Terraform:

```
# Terraform local state
*.tfstate
*.tfstate.backup

# Terraform crash logs
crash.log
crash.*.log

# Local Terraform directory
.terraform/

# Terraform plan files
*.tfplan

# Environment variable files
*.env

# Sensitive variable files
*.tfvars
*.auto.tfvars

# Provider lock file (commit this; do NOT ignore)
# .terraform.lock.hcl

# OS / editor-specific files
.DS_Store
.idea/
.vscode/
```

Best practices for Terraform file management

- **Organize files by purpose and scope** — Keep `main.tf` for core resources, `variables.tf` for inputs, `provider.tf` for providers, and `output.tf` for outputs. Use folders or modules (`/network`, `/compute`, `/security`) to separate components and promote reuse.
- **Use environment-specific variable files** — Create `.tfvars` per environment (e.g., `dev.tfvars`, `staging.tfvars`, `prod.tfvars`) and store them in an `envs/` folder.
- **Keep state files out of version control** — Never commit `.tfstate` or `.tfstate.backup`. Store state in a remote backend to enable collaboration and prevent data loss.
- **Commit the lock file for provider consistency** — Always include `.terraform.lock.hcl`. It pins provider versions so infrastructure behaves consistently across machines and environments.
- **Automate formatting, validation, and security checks** — Run `terraform fmt`, `validate`, and `plan` before committing. Add these to CI/CD and include tools like `tfsec`, `OPA`, or `Sentinel` for security and compliance.
- **Document and version your infrastructure** — Run `terraform fmt`, `validate`, and `plan` before committing. Add these to CI/CD and include tools like `tfsec`, `OPA`, or `Sentinel` for security and compliance.
- **Document and version your infrastructure** — Run `terraform fmt`, `validate`, and `plan` before committing. Add these to CI/CD and include tools like `tfsec`, `OPA`, or `Sentinel` for security and compliance.

Best practices for structuring Terraform projects

- **Use a consistent, modular layout** — Separate reusable modules from environment-specific root configurations.
- **Separate environments with dedicated variable and state** — Each env (dev, staging, prod) should have its own `.tfvars` and backend config.
- **Adopt a clear naming convention** — Use descriptive, consistent names (e.g., `aws_instance.web_server` over `aws_instance1`).
- **Keep reusable logic inside modules** — Put common components (VPCs, EC2, IAM) under `/modules`.
- **Integrate automation and version control** — Use Git for versioning, branch by environment or feature, and wire up CI/CD for plan/apply gates.

How to organize Terraform files for bigger projects

Split an oversized main.tf

Choose one approach and keep files at the root (Terraform reads only `.tf` in the current directory; subdirectories require modules).

- **By service:** `service-payments.tf`, `service-analytics.tf` — everything a service needs in one place, great for RCA.
- **By component:** `network.tf`, `compute.tf`, `databases.tf`, `security.tf` — you always know where to look.

Variables and .tfvars (loading and precedence).

Terraform automatically loads `terraform.tfvars` and any `*.auto.tfvars` files (lexical order).

Files with other names must be passed via `-var-file`. CLI flags (`-var` and `-var-file`) override auto-loaded values.

Multi-environment pattern.

Same code, different values via `.tfvars`:

```
variables-dev.tfvars
variables-qa.tfvars
variables-prod.tfvars
```

Run with the right file (and workspace if used):

```
terraform workspace select qa
terraform plan -var-file=variables-qa.tfvars
```

Using Spacelift.

Map environments to **Stacks** and attach **Contexts** for variables and credentials to avoid manual mistakes.

Subdirectories and modules.

Terraform does not automatically read `.tf` in subfolders. Put reusable logic under `/modules/` and call it from the root.

```
.
├── provider.tf  variables.tf  outputs.tf
├── network.tf   compute.tf    databases.tf    # or service-*.tf
├── variables-*.tfvars
├── modules/
│   ├── network/ (main.tf, variables.tf, outputs.tf)
│   └── compute/  (main.tf, variables.tf, outputs.tf)
```

Recommended day-to-day workflow

1. Choose a slicing approach (by service or by component).
2. Extract repeated logic into `/modules/` and reference it from root files.
3. Declare inputs in `variables.tf`, keep one `terraform.tfvars` for shared defaults, and maintain one `.tfvars` per environment.
4. Use modules for anything that is reused across services or environments.
5. Tie environments to Terraform workspaces or Spacelift Stacks so the correct variable sets apply consistently.

Best practices for managing complex IaC setups

- **Modularize** — Small, versioned modules with clear inputs/outputs. Separate platform layers from app layers.
- **Enforce security as code** — Run policy checks on every PR. Use least-privilege roles and externalize secrets.
- **Standardize via a central registry** — Publish curated modules from a CoE, provide golden templates, and clear upgrade paths.
- **Isolate environments** — Separate state per env. Drive differences through `*tfvars`. Support ephemeral preview envs.
- **Scale by region with data, not forks** — Pass region via variables and provider aliases. Enforce regional guardrails with policy.
- **Make safe delivery the default** — Treat plan as an artifact. Validate, format, and scan in CI. Use remote state with locking and drift detection.

Spacelift tips

- Use **Stack** per deployable unit or per env/region.
- Leverage **Contexts** for shared config, credentials, and policy inputs.
- Attach **Policies** to block noncompliant plans; use **Worker Pools** for isolation and control.